The actual update of the `tail` pointer has been extracted into `set_new_tail()` ❶. This uses a `compare_exchange_weak()` loop ❷ to update the `tail`, because if other threads are trying to `push()` a new node, the `external_count` part may have changed, and you don't want to lose it. However, you also need to take care that you don't replace the value if another thread has successfully changed it already; otherwise, you may end up with loops in the queue, which would be a rather bad idea. Consequently, you need to ensure that the `ptr` part of the loaded value is the same if the compare/ exchange fails. If the `ptr` is the same once the loop has exited ❸, then you must have successfully set the `tail`, so you need to free the old external counter ❹. If the `ptr` value is different, then another thread will have freed the counter, so you just need to release the single reference held by this thread ❺.

If the thread calling `push()` failed to set the `data` pointer this time through the loop, it can help the successful thread to complete the update. First off, you try to update the `next` pointer to the new node allocated on this thread ⓫. If this succeeds, you want to use the node you allocated as the new `tail` node ⓬, and you need to allocate another new node in anticipation of actually managing to push an item on the queue ⓭. You can then try to set the `tail` node by calling `set_new_tail` before looping around again ⓮.

You may have noticed that there are rather a lot of `new` and `delete` calls for such a small piece of code, because new nodes are allocated on `push()` and destroyed in `pop()`. The efficiency of the memory allocator therefore has a considerable impact on the performance of this code; a poor allocator can completely destroy the scalability properties of a lock-free container such as this. The selection and implementation of such allocators is beyond the scope of this book, but it's important to bear in mind that the only way to know that an allocator is better is to try it and measure the performance of the code before and after. Common techniques for optimizing memory allocation include having a separate memory allocator on each thread and using free lists to recycle nodes rather than returning them to the allocator.